

Parallel Blocked Matrix Multiplication using OpenMP

1. Introduction

Matrix multiplication is a core operation in scientific computing. Direct implementations are memory-intensive and suffer from poor cache utilization. To optimize, *blocked matrix multiplication* is used, which improves spatial and temporal locality. In this project, we implemented blocked matrix multiplication in C with OpenMP parallelization. Performance was evaluated for different block sizes and thread counts.

2. Methodology

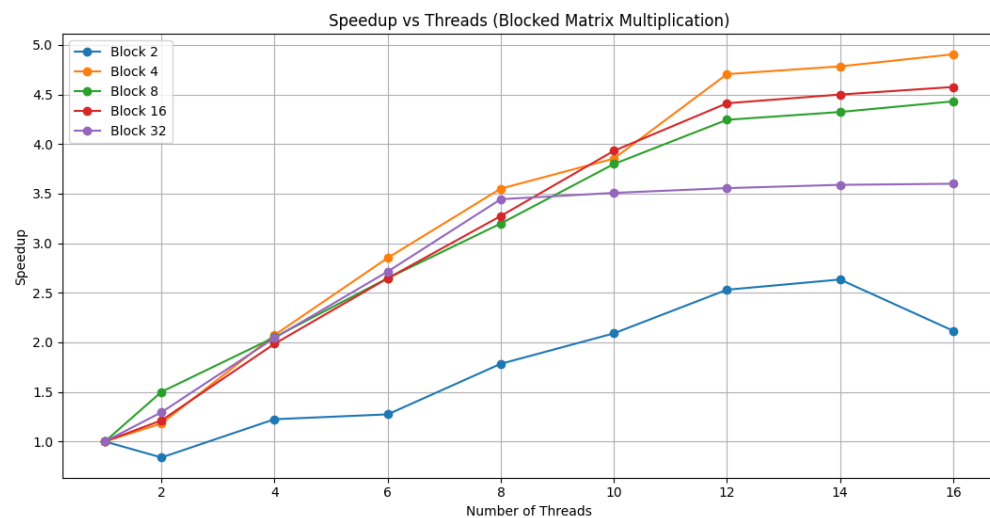
- Matrix size: 4096×4096 .
- Input: random numbers from uniform $[0,1]$.
- Block sizes tested: 2, 4, 8, 16, 32.
- Threads tested: 1, 2, 4, 6, 8, 10, 12, 14, 16.
- Timing: `gettimeofday()`; average of 5 runs.
- Metrics: Average Runtime, Speedup.

3. Results

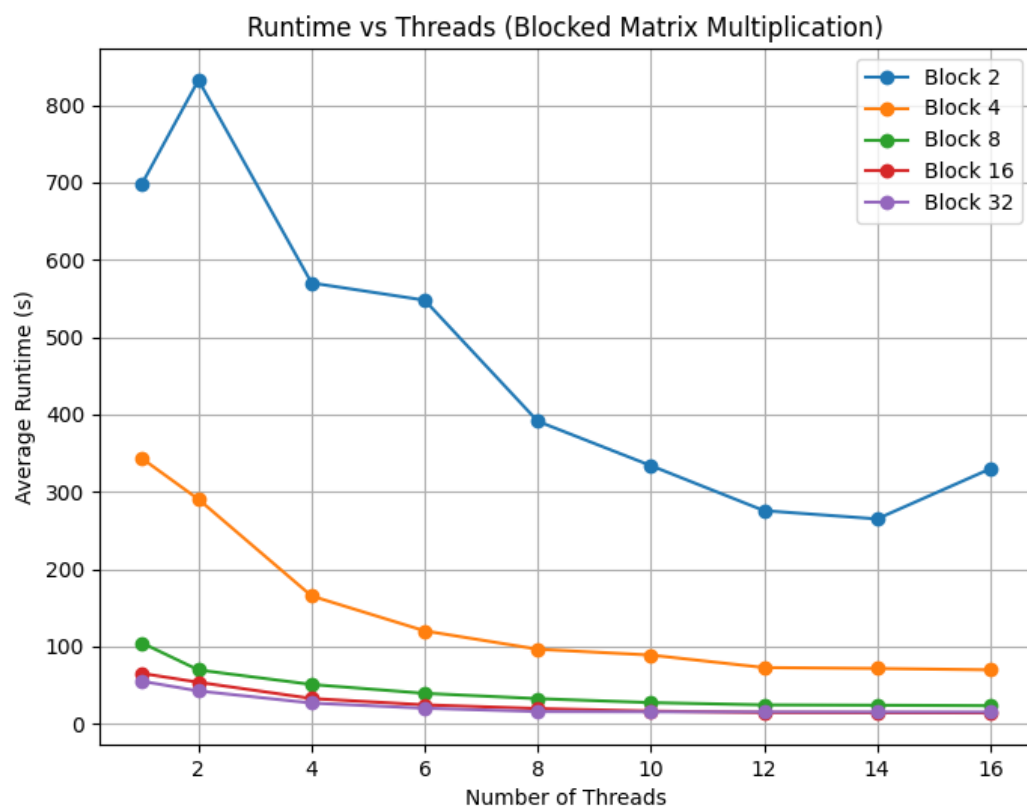
- **Block Size 2:**
Very poor scaling. At 16 threads, speedup ≈ 2.1 only. Runtime remains very high ($>250s$).
- **Block Size 4:**
Significant improvement. Runtime drops to $\sim 70s$ at 16 threads, speedup ≈ 4.9 .
- **Block Size 8:**
Strong scaling. Runtime $\sim 24s$ at 16 threads, speedup ≈ 4.4 .
- **Block Size 16:**
Best overall performance. Runtime $\sim 14.3s$ at 16 threads, speedup ≈ 4.6 .
- **Block Size 32:**
Runtime $\sim 15.3s$ at 16 threads, speedup ≈ 3.6 (performance flattens).

Graphs:

- *Speedup vs Threads* → Near-linear up to ~12 threads for block sizes 4–16.



- *Runtime vs Threads* → Clear decrease in runtime; best runtime achieved with block 16.



4. Analysis

- **Small block (2):** Overhead of loop control dominates, poor cache reuse.
- **Medium block (4–16):** Balances cache efficiency and parallelism. Block 16 gave the lowest runtime.
- **Large block (32):** Fewer blocks → less parallel granularity, threads underutilized.
- **Thread scaling:** Speedup flattens after ~12 threads, likely due to hardware core limits and memory bandwidth saturation.

5. Conclusion

- Blocked multiplication with OpenMP significantly reduces runtime compared to sequential execution.
- Block size 16 was optimal, achieving ≈ 14 s runtime and $\approx 4.6\times$ speedup on 16 threads.
- Parallel efficiency decreases with very small or very large block sizes.
- Future improvements could include NUMA-aware scheduling and SIMD vectorization.